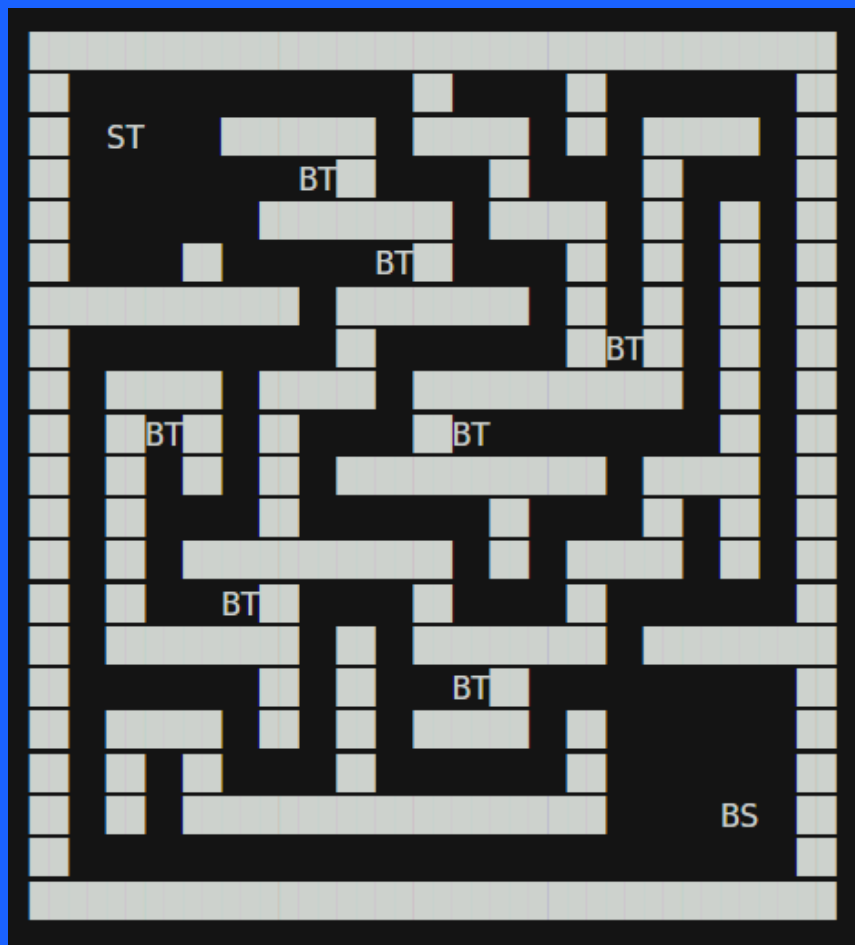


Game Design Document

Dragon Chess Maze

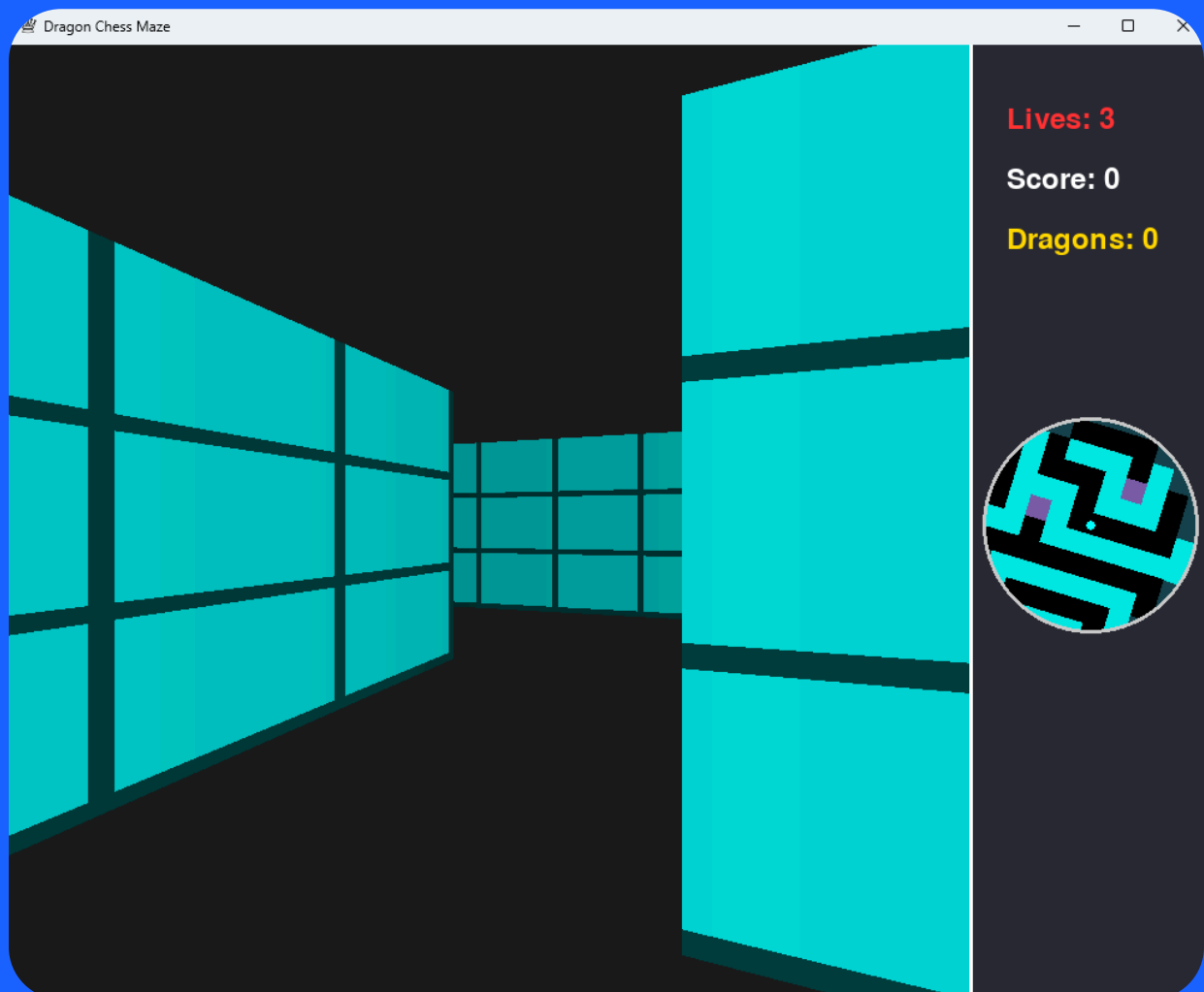
By Shyam Parikh and Samuel Boncik



Project Overview

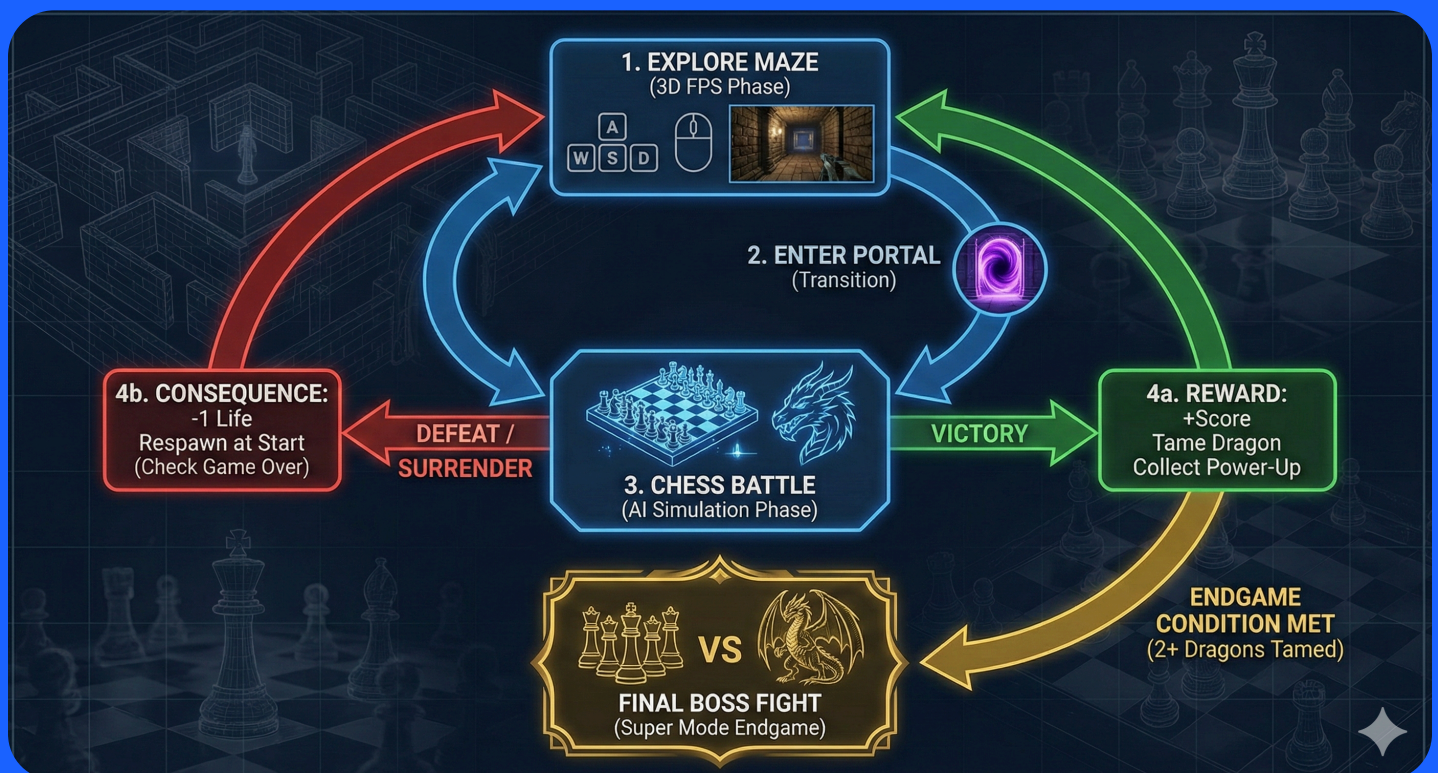
Dragon Chess Maze has been a massive learning journey. Throughout development, I explored concepts ranging from camera movement using matrices to designing a functional chess AI. The project helped me understand the importance of small implementation details — such as how a single forgotten return statement can create a very long (and sarcastically “fun”) debugging session.

Dragon Chess Maze is where I first learned object-oriented programming, GPU-based graphics rendering, and advanced gameplay architecture. The game blends exploration and simulation into a hybrid experience where strategy meets discovery.



Core Gameplay Loop

Players explore a procedurally generated maze to locate dragon portals. Entering a portal transitions the player into a chess battle. Winning battles rewards progression toward defeating the final boss. Winning battles rewards progression toward defeating the final boss.



Credits: Image made with Nano banana Pro

Exploring the Unknown

The game begins with the player navigating a mysterious, procedurally generated maze.

Maze Generation

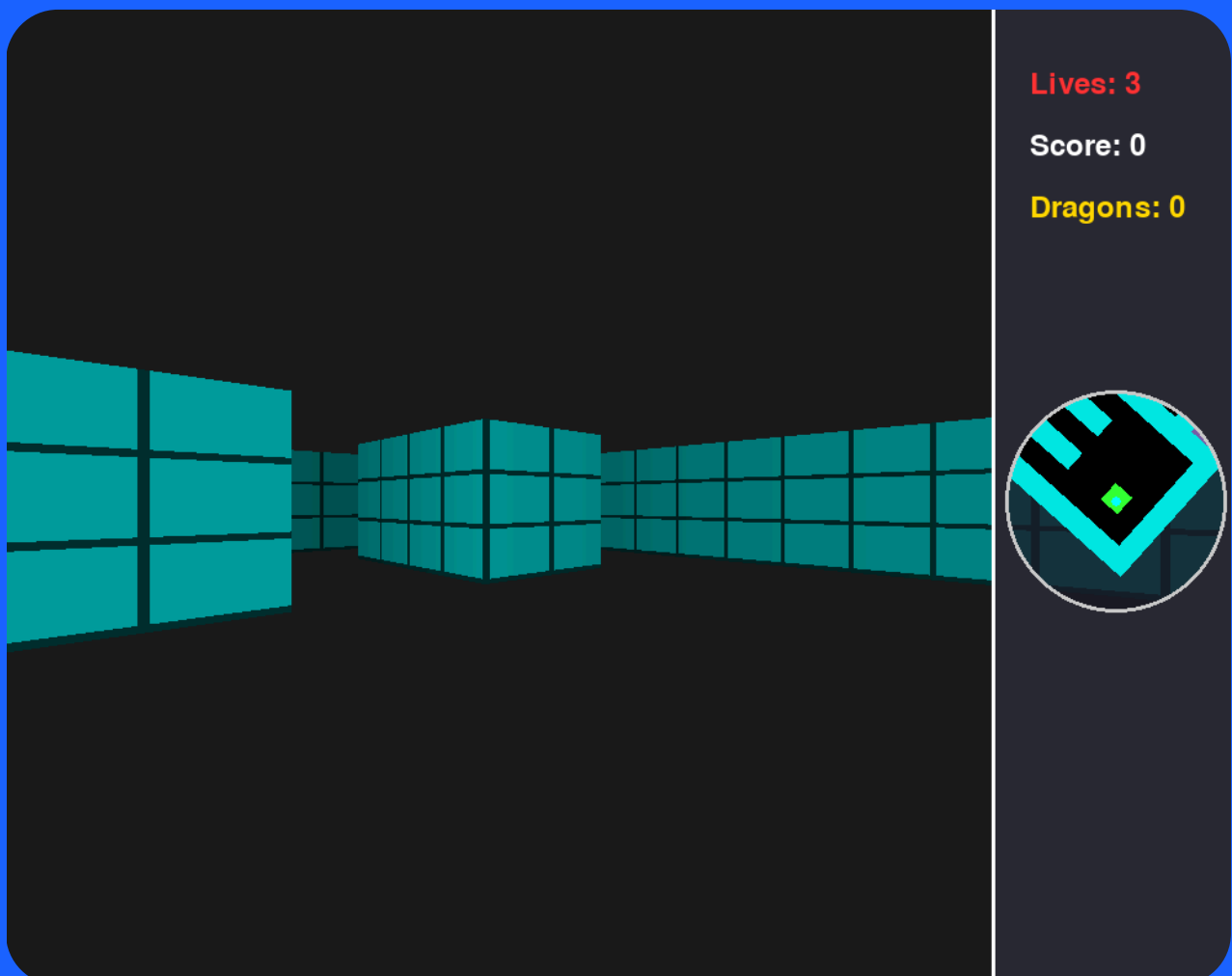
- The maze is generated using **recursive backtracking**, a depth-first search algorithm.
- Performance was optimized using **Numba (njit)** to significantly reduce generation time.

Player Movement

- Movement controls use:
 - **W and S keys**
 - **Arrow Keys**
- The **mouse controls camera perspective**, allowing full first-person navigation.

Collision System

Player movement is not purely positional. Each movement is validated through collision detection to ensure proper interaction with the environment.



Strategy and Combat

When the player enters a portal, gameplay transitions from a 3D exploration environment to a 2D chess battle.

Chess AI

- The AI opponent is powered by the classic **Minimax algorithm**.
- Performance improvements include:
 - Numba optimization
 - NumPy vectorization
 - Bitwise logic operations

Player Stakes

- Players begin with **3 lives**.
- Losing or surrendering a chess battle removes **one life**.
- Losing all three lives results in a full game restart.



RPG Progression System – "Super Mode"

One of the core progression systems in Dragon Chess Maze is the **Super Mode** mechanic.

Dragon Taming

Each dragon minion defeated increases a counter tracking total dragons conquered.

Rewards

Players earn **additional Queen pieces** in the final boss battle depending on the number of dragons defeated.

Scoring

Players receive increased scoring bonuses for successfully defeating the final boss dragon.

Lives: 3

Score: 0

Dragons: 0

Strategic Design Philosophy

The game's mechanics were influenced by performance and engagement KPIs.

Platform Focus

- PC remains one of the most profitable platforms long-term, averaging approximately **\$59,519.66** in revenue.
- As a result, the control system was optimized for keyboard and mouse input.

Genre Influence

- Action and RPG genres average over **\$54,000** in revenue.
- Simulation games show the highest engagement times, averaging nearly **60 minutes** of gameplay.
- Dragon Chess Maze combines these elements through real-time exploration and strategic chess battles.

Streaming & Engagement Value

- The first-person maze exploration creates high-intensity gameplay moments.
- Chess battles provide strategic depth that encourages player retention and replay ability.

```
def update(self, keys, grid, delta_time = 0.016):
    self.angle_mouse()
    sin_a = np.sin(self.angle_x)
    cos_a = np.cos(self.angle_x)

    speed = self.speed
    if self.speedster and keys[pg.K_LSHIFT]:
        speed = speed * 2
    actual_speed = (speed * 60.0) * delta_time

    delta_x = 0.0
    delta_z = 0.0
    if self.can_move:
        if keys[pg.K_w] or keys[pg.K_UP]:
            delta_x += actual_speed * cos_a
            delta_z += actual_speed * sin_a
        if keys[pg.K_s] or keys[pg.K_DOWN]:
            delta_x -= actual_speed * cos_a
            delta_z -= actual_speed * sin_a
        if keys[pg.K_a] or keys[pg.K_LEFT]:
            delta_x += actual_speed * sin_a
            delta_z -= actual_speed * cos_a
        if keys[pg.K_d] or keys[pg.K_RIGHT]:
```

Formatted with [Carbon](#) | [Create and share beautiful images of your source code](#)

Technical Implementation

Rendering Engine

The game uses a custom-built graphics engine powered by:

- **ModernGL**
- Custom vertex and fragment shaders

This ensures high-performance rendering while maintaining stable frame rates.

2D Overlay System

- Pygame surfaces are converted into OpenGL textures.
- This allows the chess board and UI elements to render as HUD overlays.
- Combines:
 - Pygame for UI and input handling
 - ModernGL for rendering performance

```
VERTEX_SHADER_3D = """
#version 330 core
in vec3 in_pos;
in vec3 in_instance_pos;
in float in_instance_type;

uniform mat4 m_proj;
uniform mat4 m_view;

out vec3 v_world_pos;
out float v_type;

void main() {

    vec3 world = in_pos + in_instance_pos;
    v_world_pos = world;
    v_type = in_instance_type;
    gl_Position = m_proj * m_view * vec4(world, 1.0);

}

"""
FRAGMENT_SHADER_3D = """
...
"""
```

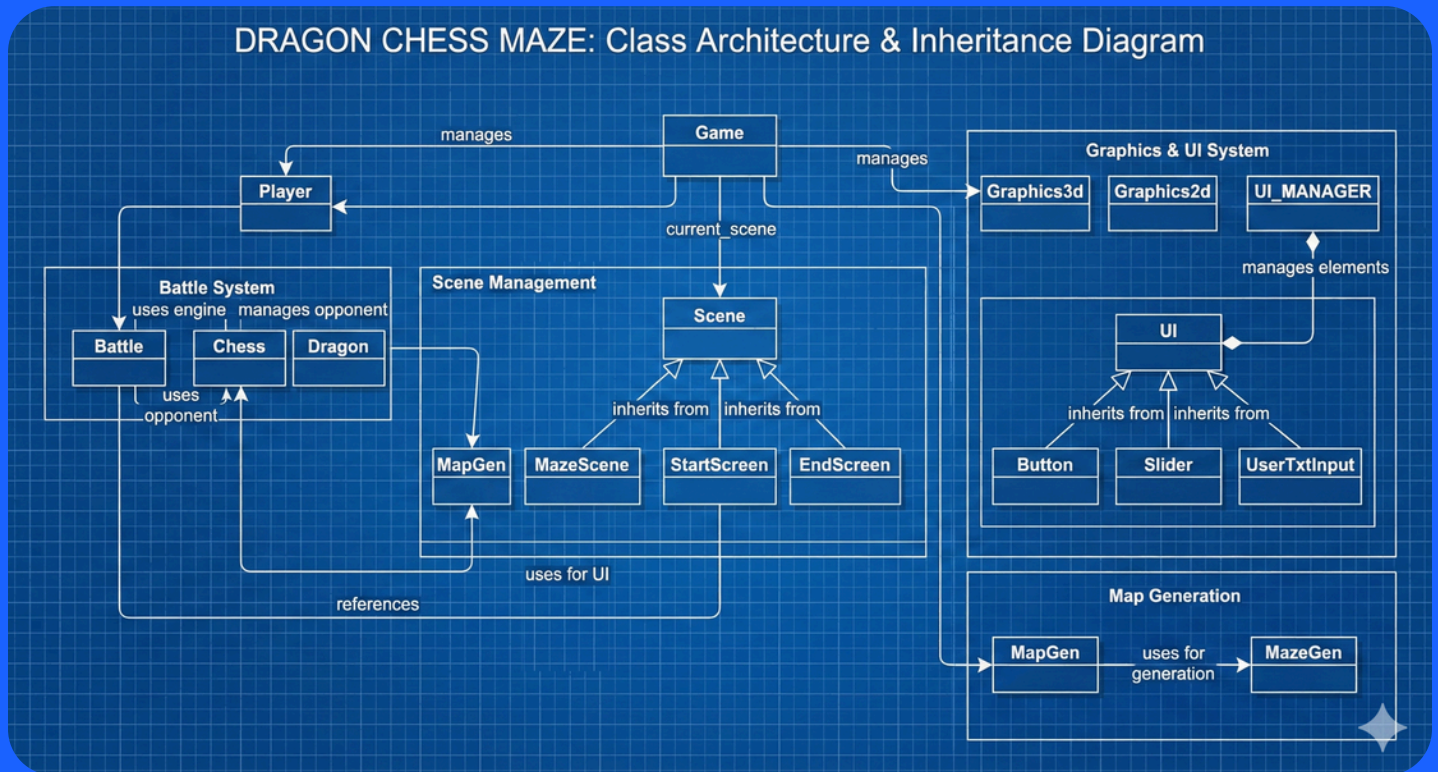
Formatted with [Carbon](#) | [Create and share beautiful images of your source code](#)

Architecture & Scene Management

The entire game is built using **Object-Oriented Programming**.

Key architectural features include:

- Scene-based state management
- Seamless transitions between:
 - 3D Maze Exploration
 - 2D Chess Battles



Credits: Image made with Nano banana Pro

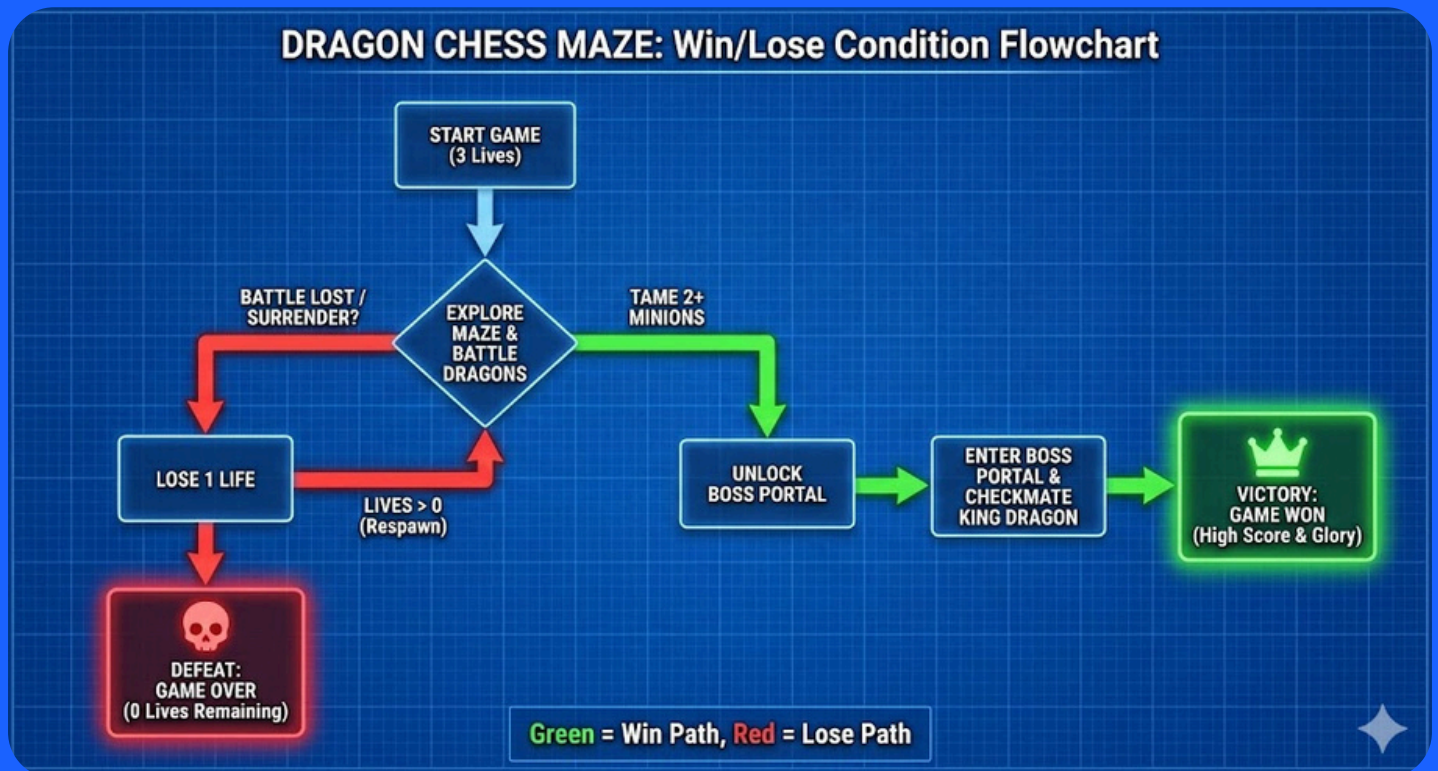
Win and Lose Conditions

Victory Condition

- Players must defeat at least **two dragon minions**.
- Once unlocked, players must locate the **Boss Portal (Red)**.
- Checkmating the boss dragon results in total victory.

Failure Condition

- Losing all three lives ends the game and forces a restart.



Credits: Image made with Nano banana Pro

KPI - Game Mapping Table

KPI	Insight	Game Mechanic	Where it Appears in Gameplay
Revenue per Platform	PC is the most profitable market, generating the highest average revenue.	Utilizing precise W / D and Mouse-Look controls.	Entire game loop and movement system (game.py, player.py).
Session Duration	Simulation games drive the longest engagement times at 59.27 minutes.	High-complexity Chess AI matches utilizing a deep Minimax algorithm with Alpha-Beta pruning.	Strategic combat phase during portal encounters (battle_scene.py, dragon.py).
Revenue per Genre	Action & RPG are top earners, making them the most viable genres for profitability.	A Hybrid Action-RPG loop that combines real-time exploration with tactical turn-based combat.	The core loop of exploring the maze to collect dragon power (maze_scene.py).
Stream Viewership	FPS & Action games capture the largest streaming audiences and highest "virality" potential.	Immersive 3D First-Person Perspective maze exploration rendered via a custom ModernGL engine.	Exploration phase while navigating the procedurally generated grid (graphics.py).
In-game Purchases	RPG players show the highest willingness to invest in power-ups and progression.	"Super Mode" Collection Logic: Defeating minion dragons grants extra Queens for the final boss encounter.	Minion portals (Type 3) and the Boss fight setup logic (maze_scene.py, chess_engine.py).
Market Trend (Growth)	The market for action-heavy, streamable content peaked between 2018–2024.	High-stakes Portal Transitions and dramatic scene shifts between 3D and 2D environments.	The transition animations and the final "Victory" vs "Game Over" logic (battle_scene.py).

Author's Notes

Developing Dragon Chess Maze has been an exciting and educational experience. While debugging was often challenging, the process was incredibly rewarding.

If anyone would like to build something similar using this project as inspiration, feel free to explore and use the code. Credit would be greatly appreciated.

Learning Resources & Inspiration

ModernGL & Graphics Programming

- <https://youtu.be/LFbePt8i0DI?si=uUN8aOSwb6jsgSu3>

Chess AI & Algorithms

- https://youtu.be/_vqIIPDR2TU?si=0uiNtLrtIJMjF8E1
- <https://youtu.be/X-e0jk4I938?si=8ku0YFi0XqQldcXE>
- <https://youtu.be/OpL0Gcfn4B4?si=18hJkN6JzBymSZAr>